LABORATORY

LAB 10

NFC Card Detection with PN532

Reading Card UIDs over UART (HSU) Using EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1 Introduction . . . 2
1.1 Lab Overview . . . 2
1.2 Objectives . . . 2
2 Background . . . 2
2.1 NFC and Contactless Identification Systems . . . 2
2.2 PN532 and the Unique Identifier (UID) . . . 3
2.3 UART/HSU Communication Between PN532 and S32K144 . . . 4
2.4 From Card Detection to UID Display . . . 4
3 Hardware Setup . . . 4
3.1 Required Hardware . . . 4
3.2 UART Pin Mapping and PN532 Connections . . . 5
3.3 Selecting HSU Mode on the Module . . . 6
4 EduFramework API . . . 7
4.1 `PN532_Begin()` . . . 7
4.2 `PN532_IsCardPresent()` . . . 7
4.3 `PN532_GetUID()` . . . 8
4.4 `PN532_End()` . . . 8
5 Lab Exercise . . . 9
5.1 Requirements . . . 9
5.2 Program . . . 9
5.3 Verification . . . 11
6 Extension . . . 11
6.1 Inspecting Card Information . . . 11
6.2 Distinguishing Cards by UID and LED Feedback . . . 12
6.3 Diagnosing Communication Status . . . 12
6.4 Reading Data Stored on a Card . . . 12
7 References . . . 12

1 Introduction

1.1 Lab Overview

Contactless identification allows a device to exchange information with a card or tagged object without an electrical contact. Near Field Communication (NFC) is a short-range wireless communication technology closely related to Radio Frequency Identification (RFID) systems operating at `13.56 MHz`. The survey by Coskun, Ozdenizci, and Ok discusses the foundations of NFC and applications such as electronic cards, electronic ticketing, and device-to-device interaction [1].

In a card-reading system, a reader communicates wirelessly with a card and forwards the acquired data to the application controller. For example, Mohandes investigated an attendance management system using NFC-enabled devices and cards in a university environment. This is an application of contactless identification, rather than a study specifically concerned with the PN532 module [2].

This lab uses a **PN532 NFC RFID Module V3** and the MaaZEDU Development Board. The PN532 communicates with the S32K144 microcontroller through UART/HSU. EduFramework provides a device-level API to initialize the module, detect ISO/IEC 14443 Type A cards, and display their UIDs on the Serial Monitor. Low-level frame processing, target selection, and card-memory access are outside the scope of the main exercise [4] [7].

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the roles of an NFC reader, a card/tag, and a unique identifier (UID) in a basic contactless identification system.
2. Identify the PN532 UART/HSU interface and explain the crossed `TX` and `RX` connections between the module and MaaZEDU.
3. Distinguish reading a UID from accessing data stored on a card, and recognize the limitations of UID-based authentication.
4. Use `PN532_Begin()`, `PN532_IsCardPresent()`, `PN532_GetUID()`, and `PN532_End()` to identify cards through EduFramework.
5. Develop and verify a program that displays hexadecimal UIDs on the Serial Monitor and test it with compatible cards/tags.

2 Background

2.1 NFC and Contactless Identification Systems

NFC relies on short-range electromagnetic interaction. In **The Survey on Near Field Communication**, Coskun and colleagues describe NFC as a short-range communication technology based on inductive coupling and supporting several operating modes. RFID and NFC share related technologies and applications; however, not every RFID card should be treated as an NFC device offering the full range of NFC communication capabilities [1].

The PN532 is a `13.56 MHz` contactless communication controller. The PN532/C1 datasheet describes its support for multiple protocols and modes, including communication with ISO/IEC 14443 Type A/MIFARE cards. Lab 10, however, addresses one specific use case: **a reader detects a Type A card and obtains its UID** [3] .

Component	Role in the Exercise
MaaZEDU / S32K144	Controls the application, receives results over UART, and displays information on the Serial Monitor.
PN532 module	Communicates wirelessly with the card and exchanges commands and responses with the MCU.
NFC card/tag	The detected object that supplies identification information.
Serial Monitor	Displays the UID obtained by the application from the PN532.

Table 10.2.1. Components of the card-identification system in Lab 10

In this lab, UART provides the communication link **between the MCU and the module**, whereas `13.56 MHz` radio communication occurs **between the PN532 and the card**. These links serve different purposes and do not operate at the same data rate [3] [4] .

2.2 PN532 and the Unique Identifier (UID)

During ISO/IEC 14443 Type A card detection, the PN532 can perform the operations required to identify and select a card. A successful `InListPassiveTarget` command can return `ATQA` , `SAK` , the UID length, and the UID bytes. These response fields are described in NXP's **PN532 User Manual** [4] .

A UID (**Unique Identifier**) is represented as a sequence of bytes. According to NXP application note AN10927, ISO/IEC 14443 Type A UIDs may be `4` , `7` , or `10 bytes` long. The same document explains that not every UID is guaranteed to be globally unique under all conditions; some products use Random IDs or Non-Unique IDs [5] .

UID Length	Designation	Notes
`4 bytes`	Single Size UID	Selected through one cascade level.
`7 bytes`	Double Size UID	Selected through two cascade levels.
`10 bytes`	Triple Size UID	Selected through three cascade levels.

Table 10.2.2. ISO/IEC 14443 Type A UID lengths according to AN10927

For example, the UID observed during the hardware test for this lab was:

```
EE 68 17 05
```

This is a **hexadecimal** representation: each pair of characters denotes one byte. The UID is printed for observation and comparison across scans. The main program does not access user memory, authentication keys, or application data stored on the card [7] .

> **WARNING**
>
> A UID can illustrate how cards are distinguished in a laboratory exercise, but **a UID alone must not be treated as proof of secure authentication**. NXP documents Random ID and Non-Unique ID cases in contactless card systems. The UID-comparison extension is therefore a data-processing demonstration, not a secure access-control solution [5] .

2.3 UART/HSU Communication Between PN532 and S32K144

The PN532 supports SPI, I2C, and High Speed UART (`HSU`) host interfaces. In UART communication, a device transmits data through `TX` and receives through `RX` ; consequently, one device's `TX` must connect to the other device's `RX` . Both devices also require a common `GND` voltage reference [3] [6] .

In EduFramework, the PN532 driver uses `Serial2` at a fixed baud rate of `115200` . `Serial1` remains dedicated to Serial Monitor output at `9600` baud, as in Lab 03. The application does not directly access S32K144 LPUART registers because UART operations are encapsulated in the driver layers [7] .

Channel	Baud Rate	Purpose
`Serial1`	`9600`	Outputs messages and UIDs to the Serial Monitor.
`Serial2`	`115200`	Provides bidirectional UART/HSU communication with the PN532 module.

Table 10.2.3. Serial channels used in the exercise

Note

`115200 baud` is the UART rate between MaaZEDU and PN532 as configured by the driver. It is neither the `13.56 MHz` NFC carrier frequency nor the wireless data rate used between the reader and the card [3] [7] .

2.4 From Card Detection to UID Display

According to the **PN532 User Manual**, `InListPassiveTarget` allows the PN532 to detect and initialize a target in passive mode. With the Type A `106 kbps` parameter, the chip performs the required operations to detect a compatible card and returns identification data to the host [4] .

The EduFramework PN532 driver encapsulates this exchange through its Easy API. `PN532_Begin()` initializes UART and the module, `PN532_IsCardPresent()` scans for a card, and `PN532_GetUID()` retrieves the UID string cached after a successful scan. `PN532_End()` releases the selected target in preparation for subsequent processing [7] .

The application data flow can be summarized as follows:

`NFC card/tag` ⇄ `PN532 (RF 13.56 MHz)` ⇄ `UART2 (HSU)` ⇄ `S32K144` → `Serial1` → `Serial Monitor` .

Frame construction, checksum verification, ACK handling, timeouts, and response parsing are implemented in the driver. The main program therefore does not need to process PN532 protocol byte arrays [4] [7] .

3 Hardware Setup

3.1 Required Hardware

The lab uses a MaaZEDU Development Board, a red PN532 NFC RFID Module V3, at least one compatible ISO/IEC 14443 Type A card or tag, jumper wires, and a USB cable. Only UART/HSU is used in the main exercise; the SPI and I2C lines are not connected to the module [6] [8] .

Hardware	Description
MaaZEDU Development Board	Board based on the S32K144 microcontroller with an external UART interface.
PN532 NFC RFID Module V3	PN532-based card reader with an onboard PCB antenna and HSU interface selection.
NFC card/tag	A compatible card or key fob used to verify UID detection.
Jumper Wires	Four wires for power, GND, UART TX, and UART RX.
USB Cable	Connects MaaZEDU to a computer for programming and Serial Monitor access.

Table 10.3.1. Hardware required for the exercise

3.2 UART Pin Mapping and PN532 Connections

In the EduFramework configuration used for this exercise, `Serial2` uses the S32K144 `UART2_TX` and `UART2_RX` signals. MaaZEDU's transmit pin connects to the PN532 receive pin, and vice versa. `Serial1` is reserved for the Serial Monitor and is not connected to the PN532 module [7] [8] .

Component	Logical Pin	MCU Pin	Function
PN532 `RXD`	`UART2_TX`	`PTD7`	Data transmitted from MaaZEDU to the PN532 module.
PN532 `TXD`	`UART2_RX`	`PTD6`	Data transmitted from the PN532 module to MaaZEDU.

Table 10.3.2. UART2 signal mapping used in Lab 10

PN532 Module Pin	MaaZEDU	Function
`VCC`	`3.3V`	Supplies power to the module.
`GND`	`GND`	Provides a common voltage reference.
`RXD` / `RX`	`UART2_TX`	Receives commands from the S32K144.
`TXD` / `TX`	`UART2_RX`	Sends responses to the S32K144.

Table 10.3.3. PN532 module connections in UART/HSU mode

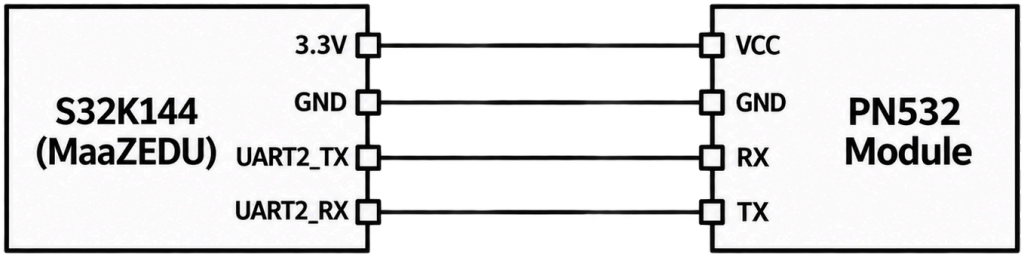


Figure 10.3.1. Wiring diagram of the PN532 UART/HSU module and MaaZEDU Development Board

3.3 Selecting HSU Mode on the Module

The PN532 V3 module has two interface-selection switches. According to the **Elechouse PN532 NFC RFID Module User Guide**, `HSU` mode is selected when both switches are `OFF`; other settings select I2C or SPI. On this module version, the HSU pins share physical connector positions with the I2C pins, while their UART signal labels are printed on the back of the PCB [6].

Mode	Channel 1	Channel 2
`HSU / UART`	`OFF`	`OFF`
`I2C`	`ON`	`OFF`
`SPI`	`OFF`	`ON`

Table 10.3.4. Interface-selection switch settings for PN532 Module V3

> **WARNING**
>
> Disconnect power before changing wiring or switch settings. Check the `TXD` and `RXD` markings on the **back** of the V3 module; do not interpret `SDA` and `SCL` printed on the front as UART signal names. Supply `3.3 V` as verified with the MaaZEDU configuration. When using another module version, consult its corresponding hardware guide [6].

4 EduFramework API

EduFramework provides straightforward functions for obtaining card information without requiring an application to declare a `PN532_Card_t` structure or parse protocol frames. The four **Easy API** functions used in the main exercise are `PN532_Begin()`, `PN532_IsCardPresent()`, `PN532_GetUID()`, and `PN532_End()` [7].

4.1 `PN532_Begin()`

`PN532_Begin()` initializes `Serial2` at `115200 baud`, wakes the module through HSU, checks the chip firmware version, and performs the necessary configuration for card-reading mode. It takes no parameters; the application only needs to check the initialization result [7].

`PN532_Begin`

Initialize UART/HSU and prepare the PN532 for card detection.

Syntax

```
PN532_Begin();
```

Return value

`PN532_STATUS_OK` on successful initialization; a `PN532_STATUS_*` error status if initialization cannot be completed.

Example:

```
if (PN532_STATUS_OK == PN532_Begin())
{
    Serial1_println("PN532 ready.");
}
```

Detailed error statuses can be converted into text using `PN532_StatusString()`. The main exercise only reports whether initialization succeeds or fails.

4.2 `PN532_IsCardPresent()`

`PN532_IsCardPresent()` requests the detection of one ISO/IEC 14443 Type A card. When a card is found, the driver caches its identification information; therefore, `PN532_GetUID()` can be called immediately after this function returns `true` [7].

`PN532_IsCardPresent`

Scan for a compatible card and cache its information after successful detection.

Syntax

```
PN532_IsCardPresent();
```

Return value

`true` if a card is detected and its information is read successfully; `false` if no card is found or communication fails.

Example:

```
if (true == PN532_IsCardPresent())
{
    Serial1_println("Card detected.");
}
```

Note

`PN532_IsCardPresent()` performs an actual communication transaction and may wait for a response from the module. A `false` result does not distinguish the absence of a card from a communication error. The detailed cause can be examined using `PN532_GetLastError()` in the Extension section [7] .

4.3 `PN532_GetUID()`

`PN532_GetUID()` does not initiate another scan. It only returns the UID string cached during the most recent successful card detection. The driver formats the UID bytes as space-separated hexadecimal pairs [7] .

`PN532_GetUID`

Retrieve the current card UID as a hexadecimal string.

Syntax

```
PN532_GetUID();
```

Return value

A UID string such as `EE 68 17 05` if the cached data are valid; an empty string if no valid UID is available in the driver cache.

Example:

```
Serial1_print("UID: ");
Serial1_println(PN532_GetUID());
```

The returned string is managed by the driver; no UID array declaration or per-byte conversion is required. The application uses the string for display and does not modify the data through the returned pointer [7] .

4.4 `PN532_End()`

`PN532_End()` releases the selected target and clears the card information cached by the Easy API. It does not power down the PN532 module; the application can continue scanning in the next loop iteration [7] .

`PN532_End`

Release the selected card after its UID has been processed.

Syntax

```
PN532_End();
```

Return value

`true` if the target is released successfully; `false` if there is no valid target or the operation fails.

Example:

```
(void)PN532_End();
```

The main exercise calls this function after printing the UID. If a card remains within the antenna field, a subsequent scan may detect it again, so the same UID can appear repeatedly on the Serial Monitor.
The `Serial1_begin()`, `Serial1_print()`, and `Serial1_println()` functions were introduced in Lab 03. Lab 10 reuses these APIs for output and does not repeat their detailed syntax documentation.

5 Lab Exercise

5.1 Requirements

Develop an application that detects cards through the PN532 and displays their UIDs on the Serial Monitor. The program must meet these requirements:

- Use the PN532 V3 module in `HSU` mode, connected to `UART2_TX` and `UART2_RX`.
- Initialize `Serial1` at `9600 baud` and display startup and initialization messages.
- Enter the scanning loop only after `PN532_Begin()` succeeds.
- When a compatible card is detected, retrieve its UID through the Easy API and print it in hexadecimal format.
- Release the selected target after processing its UID and continue waiting for the next scan.

The exercise does not require creating data structures, manipulating UID byte arrays, or directly implementing PN532 protocol commands. The white card and NFC key fob supplied with the module can be used for verification if they are compatible with the ISO/IEC 14443 Type A mode selected by the driver [7] .

5.2 Program

Implement the following program, which has been verified on the hardware, in `src/main.c` :

```
#include "Arduino.h"
#include "pn532.h"

int main(void)
{
    setup();

    Serial1_begin(9600U);
    Serial1_println("PN532 NFC Card Detection");

    if (PN532_STATUS_OK == PN532_Begin())
    {
        Serial1_println("PN532 ready.");
    }
    else
    {
        Serial1_println("PN532 initialization failed.");

        while (1)
        {
            delay(1000U);
        }
    }

    while (1)
    {
        if (true == PN532_IsCardPresent())
        {
            Serial1_print("UID: ");
            Serial1_println(PN532_GetUID());

            (void)PN532_End();
            delay(500U);
        }

        delay(100U);
    }

    return 0;
}
```

Listing 10.5.1. PN532 NFC card detection and UID display program

`setup()` prepares the underlying EduFramework components, and `Serial1_begin(9600U)` opens the logging channel. `PN532_Begin()` automatically initializes `Serial2` at `115200 baud` and prepares the PN532; therefore, the main program does not call `Serial2_begin()` separately [7] .

Within the loop, `PN532_IsCardPresent()` attempts to detect a card. After successful detection, `PN532_GetUID()` returns the formatted UID string for display. The program calls `PN532_End()` to release the target, waits `500 ms` , and then continues the loop after an additional `delay(100U)` .

Note

*`delay(100U)` introduces a wait **after** a loop iteration; it does not guarantee that the PN532 scans exactly every `100 ms`. Scanning involves UART/HSU communication, response waiting, and driver timeouts, so the interval between displayed readings also depends on the communication transaction [7] .*

5.3 Verification

Build and upload the program to MaaZEDU, then open the Serial Monitor at `9600 baud`. Check for `PN532 ready.` to confirm successful module initialization. Present a card within the antenna field and observe its UID on the Serial Monitor. Another card can be tested to compare the returned identification values.
The following log was recorded during the actual Lab 10 hardware test:

```
PN532 NFC Card Detection
PN532 ready.
UID: EE 68 17 05
UID: EE 68 17 05
UID: EE 68 17 05
UID: EE 68 17 05
UID: EE 68 17 05
```

Repeated display of the same UID indicates that the application can continue detecting a card that remains in the antenna field. The value `EE 68 17 05` belongs to the card used in this test; it is not a fixed UID of the PN532 module. The UID observed with another card depends on that card [5] [7] .
Check the following during verification:

- The application displays `PN532 ready.` after successful initialization.
- The UID is displayed as hexadecimal byte pairs separated by spaces.
- Moving the card away from the antenna stops new UID output; presenting it again allows another reading.
- If initialization fails, the application reports the failure and does not scan for cards.

EXPECTED RESULT

The PN532 operates in UART/HSU mode and initializes successfully. When a compatible ISO/IEC 14443 Type A card enters the reading field, the Serial Monitor displays its UID in hexadecimal format. The application prints a UID only after successful card detection and remains ready for subsequent scans.

6 Extension

The main exercise only detects a card and prints its UID. The following optional tasks introduce additional PN532 and EduFramework capabilities without changing the mandatory Lab 10 program [4] [7] .

6.1 Inspecting Card Information

After `PN532_IsCardPresent()` succeeds, the driver can provide the UID length, `ATQA`, and `SAK` of the detected card through `PN532_GetUIDLength()`, `PN532_GetATQA()`, and `PN532_GetSAK()`. These fields are associated with the Type A initialization and selection process described in NXP's PN532 documentation [4] [7] .
Additionally, `PN532_GetCardType()` returns a **heuristic** card-type label based on the available identification fields. This label is not conclusive evidence of the exact chip type or card capabilities.
Extension task: Display the UID length and the preliminary card-type label after a scan. Compare the information obtained from two different cards or tags.

6.2 Distinguishing Cards by UID and LED Feedback

A previously observed UID can be stored and compared with the string returned by `PN532_GetUID()`. The application can display `MATCH` when the values agree and `NOT MATCH` otherwise. The green and red MaaZEDU LEDs can provide visual feedback using the Digital Output functions introduced in Labs 01 and 02. Mohandes's study of NFC-based attendance management illustrates the broader concept of using card information in a management application [2] .

Extension task: Read and store a reference card UID, compare it with subsequent readings, and control the LEDs according to the result. Do not regard UID comparison as secure authentication: NXP documents Random IDs and Non-Unique IDs, while practical systems require suitable authentication and security measures [5] .

6.3 Diagnosing Communication Status

When `PN532_IsCardPresent()` returns `false`, the Boolean value alone cannot identify the cause. EduFramework provides `PN532_GetLastError()` to retrieve the latest operation status, `PN532_StatusString()` to convert it into a readable label, and `PN532_GetDiagnostics()` to inspect response-wait stages or received UART bytes [7] .

Extension task: Distinguish the absence of a card from a timeout or an invalid response. Add diagnostic output only after the basic UID reading application operates reliably.

6.4 Reading Data Stored on a Card

The PN532 supports data exchange with certain MIFARE card families. In EduFramework, reading a MIFARE Classic block requires appropriate authentication before calling `PN532_ReadBlock()`; the retrieved data can then be obtained using `PN532_GetBlockHex()`. The driver also providves separate operations for Ultralight/NTAG cards, which are not used in the main exercise [4] [7] .

> **WARNING**
>
> Do not write blocks, sector trailers, lock pages, OTP pages, or configuration pages without first identifying the card type and the meaning of each memory region. Write operations may change data or configuration irreversibly. This extension is limited to examining read operations on cards that the user is authorized to access [7] .

7 References

[1] Vedat Coskun, Busra Ozdenizci and Kerem Ok, *The Survey on Near Field Communication* , Sensors, 15(6), 13348–13405, 2015.

[2] Mohamed A. Mohandes, *Class Attendance Management System Using NFC Mobile Devices* , Intelligent Automation and Soft Computing, 23(2), 251–259, 2017.

[3] NXP Semiconductors, *PN532/C1 - Near Field Communication (NFC) Controller* , PN532_C1, Rev. 3.6, 2017.

[4] NXP Semiconductors, *PN532 User Manual* , UM0701-02, Rev. 02, 2007.

[5] NXP Semiconductors, *MIFARE Product and Handling of UIDs* , AN10927, Rev. 4.1, 2018.

[6] Elechouse, *PN532 NFC RFID Module User Guide* , Rev. Version 3.

[7] EduFramework, *PN532 Device API and UART/HSU Driver* , EduFramework Source Code.

[8] FPT Software, *MaaZEDU Development Board Guide* , Rev. 1.0.